

Relazione tecnica di progetto

Nutrition Planner

Stima delle calorie bruciate tramite Machine Learning, inferenza Prolog e generazione di un piano alimentare giornaliero o settimanale

Campo	Valore
Autore	Termine Christian Domenico
Docente	Prof. Nicola Fanizzi
Anno accademico	2025/2026
Linguaggi e strumenti	Python, scikit-learn, pandas, joblib, Prolog, Sphinx
Versione documento	Aggiornata al 24 giugno 2026

Indice

- 1. Introduzione
 - 1.1 Argomenti di Interesse
 - 1.2 Librerie Utilizzate
- 2. Creazione del dataset e struttura del progetto
 - 2.1 Struttura e artefatti prodotti
 - 2.2 Creazione del dataset
 - 2.3 Pulizia e Preprocessing dei dati
 - 2.4 Dataset alimentare
- 3. Apprendimento Supervisionato
 - 3.1 Addestramento dei Modelli
 - 3.2 Regressione Lineare
 - 3.3 Random Forest
 - 3.4 Gradient Boosting
 - 3.5 Grafici
 - 3.6 Stima Calorie Bruciate
- 4. Apprendimento Probabilistico
- 5. Rappresentazione della Conoscenza - KB
- 6. Modulo principale predict_models.py
- 7. Piano alimentare e nutrition_planner.py
 - 7.1 Input richiesti
 - 7.2 Formule principali
 - 7.3 Generazione dei pasti
 - 7.4 Bilanciamento del piano
 - 7.5 Esempio di output
 - 7.6 Piano settimanale
 - 7.7 Limiti e possibili estensioni
- 8. Dipendenze e Documentazione
 - 8.1 Esecuzione e test
 - 8.2 Aggiornamento dei grafici
 - 8.3 Documentazione HTML
 - 8.4 Documentazione in Prolog
 - 8.5 Dipendenze principali
 - 8.6 Installazione delle Dipendenze
- 9. Considerazioni finali
- 10. Riferimenti Bibliografici

Allineamento con la documentazione originale

Questa versione mantiene la struttura concettuale della documentazione originale del progetto sulle calorie bruciate e la amplia con le parti aggiunte nel nuovo progetto: piano alimentare, input di composizione corporea, alternative giornaliere, piano settimanale e documentazione HTML aggiornata.

Voce dell'indice originale	Presenza nella versione aggiornata	Aggiornamento nel nuovo progetto
Introduzione, argomenti e librerie	Mantenuta	Aggiornata con nutrition_planner, Sphinx separato e dataset alimentare.
Creazione dataset e preprocessing	Mantenuta	Aggiunto dataset alimenti.csv e spiegazione del suo ruolo.
Apprendimento supervisionato	Mantenuta	Riorganizzata con sottosezioni originali e metriche dei modelli.
Apprendimento probabilistico	Mantenuta	Conservata la parte Bayesian Ridge e il riferimento a SGDClassifier scartato.
Rappresentazione della Conoscenza - KB	Mantenuta	Aggiunti esempio pratico e ruolo nel flusso Python.
Dipendenze e documentazione	Mantenuta	Separati requirements.txt e requirements-docs.txt.
Piano alimentare	Nuova sezione	Descrive BMR, TDEE, macro, alternative e piano settimanale.

1. Introduzione

Il progetto nasce con l'obiettivo di stimare le calorie bruciate durante una sessione di allenamento e di trasformare questa informazione in un supporto piu' completo per l'utente. La versione aggiornata integra alla parte predittiva originale un modulo nutrizionale in grado di generare un piano alimentare giornaliero o settimanale a partire da peso, altezza, sesso, eta', ore di allenamento settimanali, massa grassa e massa magra.

Il sistema combina tre componenti: modelli di regressione per la stima delle calorie, una Knowledge Base Prolog per l'inferenza su allenamento e intensita', e un dataset alimentare usato per costruire pasti coerenti con il fabbisogno calorico stimato.

1.1 Argomenti di Interesse

- Apprendimento supervisionato con Regressione Lineare, Random Forest e Gradient Boosting.
- Apprendimento probabilistico con Bayesian Ridge e intervallo di confidenza.
- Rappresentazione della conoscenza tramite Prolog.
- Calcolo nutrizionale tramite BMR, TDEE, macronutrienti e distribuzione dei pasti.
- Documentazione tecnica tramite README, Sphinx e relazione PDF aggiornata.

1.2 Librerie Utilizzate

Libreria	Utilizzo nel progetto
pandas	Caricamento e manipolazione dei dataset CSV.
joblib	Salvataggio e caricamento dei modelli addestrati e degli scaler.
scikit-learn	Training, regressione, grid search, metriche e standardizzazione.
numpy	Calcolo numerico e metriche di errore.
matplotlib	Generazione dei grafici di valutazione.
json	Salvataggio strutturato degli iperparametri migliori.
os / pathlib	Gestione dei percorsi, cartelle e file del progetto.
pyswip	Integrazione tra Python e Prolog per interrogare la KB.
Sphinx	Generazione della documentazione HTML a partire dai file RST.
ReportLab / pypdf	Generazione e verifica della relazione PDF aggiornata.

2. Creazione del dataset e struttura del progetto

Come nella documentazione originale, il dataset principale del progetto proviene dalla piattaforma Kaggle e si trova nella cartella dataset con il nome `gym_members_exercise_tracking.csv`. A questa base e' stato aggiunto il dataset alimentare `alimenti.csv`, necessario per il nuovo modulo nutrizionale. La struttura del progetto separa addestramento, inferenza, dataset, base di conoscenza e documentazione. Questa separazione rende piu' semplice aggiornare un singolo componente senza modificare l'intero flusso.

Percorso	Ruolo
predict_models.py	Script principale: predizioni, Prolog e piano alimentare.
nutrition_planner.py	Modulo nutrizionale introdotto nella versione aggiornata.
dataset/gym_members_exercise_tracking.csv	Dataset per il training delle calorie bruciate.
dataset/alimenti.csv	Dataset degli alimenti usato per generare i pasti.
kb/kb.pl	Knowledge Base Prolog con fatti e regole.
apprendimento_supervisionato/	Training dei modelli supervisionati e grafici.
apprendimento_probabilistico/	Training Bayesian Ridge e grafici probabilistici.
docs/	Sorgenti Sphinx della documentazione HTML.
docs/_build/html/	Build HTML navigabile aggiornata.
documentazione.pdf	Relazione principale del progetto.

2.1 Struttura e artefatti prodotti

Il progetto produce diversi artefatti: modelli addestrati in formato .pkl, scaler, mapping dei workout, grafici di valutazione, tabelle degli iperparametri, documentazione HTML e relazione PDF. Questa distinzione e' utile per capire quali file vengono aggiornati dal training e quali vengono invece usati durante la predizione.

2.2 Creazione del dataset

Dataset degli allenamenti

Il dataset principale contiene 973 osservazioni e 15 colonne relative a utenti, parametri fisiologici e sessioni di allenamento. Il target predittivo e' Calories_Burned, mentre le feature usate nel training sono Gender, Workout_Type, Session_Duration_(hours), Weight_(kg), Age e Avg_BPM.

Campo	Descrizione
Age	Eta' dell'utente.
Gender	Genere dell'utente, codificato come variabile numerica.
Weight (kg)	Peso in chilogrammi.
Height (m)	Altezza in metri.
Max_BPM	Frequenza cardiaca massima registrata durante l'allenamento.
Avg_BPM	Frequenza cardiaca media durante l'allenamento.
Resting_BPM	Frequenza cardiaca a riposo.
Session_Duration (hours)	Durata della sessione in ore.
Workout_Type	Tipologia di allenamento.

Campo	Descrizione
Calories_Burned	Variabile target: calorie bruciate.
Fat_Percentage	Percentuale di massa grassa, utile anche per la parte nutrizionale.
Water_Intake (liters)	Quantita' di acqua assunta durante la sessione.
Workout_Frequency (days/week)	Frequenza degli allenamenti settimanali.
Experience_Level	Livello di esperienza dell'utente: principiante, intermedio o avanzato.
BMI	Indice di massa corporea calcolato a partire da peso e altezza.

2.3 Pulizia e Preprocessing dei dati

La pulizia dei dati e' realizzata nel file dataset/dataset_utils.py. I passaggi principali riprendono la documentazione originale: rinomina delle colonne, conversione delle variabili categoriche e standardizzazione delle feature numeriche.

Passaggio	Descrizione
Rinomina colonne	Gli spazi nei nomi delle colonne vengono sostituiti con underscore, cosi' le feature risultano piu' facili da usare nel codice.
Workout_Type	La colonna testuale viene convertita in categoria numerica. La mappatura viene salvata in workout_mapping.pkl per riconvertire i codici.
Gender	Il genere viene mappato in valori numerici: Male = 0, Female = 1.
Standardizzazione	StandardScaler porta le feature su scala comune, con media 0 e deviazione standard 1, evitando che variabili con valori piu' grandi dominino il modello.

2.4 Dataset alimentare

La versione aggiornata introduce il file dataset/alimenti.csv. Il file non e' un Excel, ma un CSV apribile anche con Excel. Contiene 35 alimenti classificati per categoria e pasto compatibile. Ogni alimento riporta calorie, proteine, carboidrati e grassi per 100 grammi.

Colonna	Significato
alimento	Nome dell'alimento.
categoria	proteina, carboidrato, grasso, misto o verdura.
pasti	Pasti compatibili separati da , ad esempio colazione spuntino.
kcal_100g	Calorie per 100 grammi.
proteine_100g	Proteine per 100 grammi.
carboidrati_100g	Carboidrati per 100 grammi.
grassi_100g	Grassi per 100 grammi.

```
alimento, categoria, pasti, kcal_100g, proteine_100g, carboidrati_100g, grassi_100g
Avena, carboidrato, colazione|spuntino, 389, 16.9, 66.3, 6.9
Petto di pollo, proteina, pranzo|cena, 165, 31.0, 0.0, 3.6
```

3. Apprendimento Supervisionato

La parte supervisionata addestra tre modelli di regressione: Linear Regression, Random Forest Regressor e Gradient Boosting Regressor. I dati vengono divisi in training set e test set con `test_size` pari a 0.2 e `random_state` pari a 42. Le feature vengono standardizzate con `StandardScaler` prima dell'addestramento.

Modello	Scopo	Note progettuali
Linear Regression	Baseline interpretabile.	Modello semplice usato come riferimento.
Random Forest	Regressione non lineare.	Ottimizzato tramite <code>GridSearchCV</code> .
Gradient Boosting	Ensemble sequenziale.	Ottimizzato tramite <code>GridSearchCV</code> .

3.1 Addestramento dei Modelli

Dopo la preparazione iniziale, vengono selezionate le feature considerate piu' influenti nella predizione delle calorie bruciate: `Gender`, `Workout_Type`, `Session_Duration_(hours)`, `Weight_(kg)`, `Age` e `Avg_BPM`. La variabile target e' `Calories_Burned`. Il dataset viene diviso in training set, pari all'80% dei dati, e test set, pari al 20%.

3.2 Regressione Lineare

3.2.1 Scelte progettuali

La Regressione Lineare viene usata come primo approccio e come baseline interpretabile. Il modello cerca una relazione lineare tra le feature dell'utente e dell'allenamento e il target `Calories_Burned`. Non e' stata applicata una `Grid Search` perche' il modello ha pochi iperparametri regolabili rispetto a Random Forest e Gradient Boosting.

3.2.2 Valutazione del modello

Set	MAE	RMSE	Interpretazione
Test	29.32	39.47	Errore medio coerente con la baseline.
Training	29.90	39.49	Valori molto simili al test set.

3.2.3 Overfitting

Il confronto tra training e test suggerisce basso rischio di overfitting. I valori di errore indicano pero' che la relazione tra feature e calorie non e' completamente lineare, motivo per cui vengono valutati modelli ensemble piu' complessi.

3.2.4 Risultati ottenuti

La Regressione Lineare fornisce una baseline utile, ma i risultati ottenuti mostrano che modelli piu' complessi riescono a cogliere meglio la relazione tra parametri fisiologici, allenamento e calorie bruciate.

3.3 Random Forest

Random Forest e' un modello ensemble basato su piu' alberi decisionali. Utilizza il bagging per addestrare alberi su sottoinsiemi diversi del dataset e combina le predizioni per ridurre varianza e

instabilita'.

3.3.1 Iperparametri di Random Forest

Iperparametro	Significato
n_estimators	Numero di alberi nella foresta.
max_depth	Profondita' massima degli alberi.
min_samples_split	Campioni minimi necessari per dividere un nodo.
min_samples_leaf	Campioni minimi richiesti in ogni foglia.
max_features	Numero massimo di feature considerate a ogni split.
bootstrap	Uso o meno del campionamento con sostituzione.

3.3.2 Ottimizzazione degli Iperparametri con Grid Search e Cross Validation

L'ottimizzazione avviene tramite GridSearchCV con 5-fold Cross Validation. La Grid Search testa combinazioni di iperparametri, mentre la Cross Validation suddivide i dati in fold per ottenere una stima piu' robusta delle prestazioni. I risultati vengono salvati in CSV nella cartella iperparametri/tabelle, mentre i parametri migliori vengono salvati in JSON.

3.3.3 Creazione della Tabella dei Risultati

Come nella documentazione originale, i risultati della ricerca degli iperparametri vengono organizzati in tabelle CSV, cosi' da poter consultare in modo trasparente le combinazioni provate e confrontare le metriche ottenute.

3.3.4 Overfitting nel Random Forest e risultati ottenuti

Set	MAE	RMSE	Nota
Test	54.08	75.09	Errore piu' alto rispetto al training.
Training	36.50	51.12	Indica un certo grado di overfitting.

3.4 Gradient Boosting

Gradient Boosting e' un modello ensemble di boosting: costruisce alberi sequenzialmente, facendo in modo che ogni nuovo albero corregga gli errori commessi dai precedenti. Rispetto a Random Forest, l'apprendimento e' quindi progressivo e orientato alla riduzione degli errori residui.

3.4.1 Iperparametri di Gradient Boosting

Iperparametro	Significato
n_estimators	Numero di alberi nella sequenza.
max_depth	Profondita' massima degli alberi.
learning_rate	Contributo di ogni albero alla predizione finale.
subsample	Frazione di dati usata a ogni iterazione.
min_samples_split	Campioni minimi richiesti per dividere un nodo.

3.4.2 Ottimizzazione degli Iperparametri con Grid Search e Cross Validation

Anche per Gradient Boosting e' stata applicata una Grid Search con Cross Validation, valutando combinazioni di learning_rate, profondita', numero di stimatori e sottocampionamento.

3.4.3 Creazione della tabella dei risultati

Le tabelle degli iperparametri permettono di documentare in modo riproducibile quali configurazioni sono state testate e quali valori hanno prodotto le metriche migliori.

3.4.4 Valutazione del modello

Set	MAE	RMSE	Nota
Test	10.40	13.86	Migliori prestazioni tra i modelli supervisionati.
Training	4.14	5.25	Errore molto basso sul training set.

3.4.5 Risultati ottenuti

La documentazione originale evidenziava Gradient Boosting come modello particolarmente efficace per questo dataset. La versione aggiornata mantiene questa interpretazione e continua a usarlo nel confronto delle predizioni in predict_models.py.

3.5 Grafici

Modello	Iperparametri principali
Random Forest	bootstrap=False, max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=20, n_estimators=300
Gradient Boosting	learning_rate=0.1, max_depth=3, min_samples_split=5, n_estimators=300, subsample=0.8

I grafici nella cartella apprendimento_supervisionato/grafici vengono aggiornati solo quando si rilancia lo script di training supervisionato.

3.6 Stima Calorie Bruciate

La stima delle calorie bruciate avviene nel file predict_models.py: i dati inseriti dall'utente vengono trasformati nello stesso formato usato durante il training, scalati con lo scaler salvato e passati ai modelli caricati da file .pkl. L'output confronta Regressione Lineare, Random Forest, Gradient Boosting e Bayesian Ridge.

4. Apprendimento Probabilistico

La componente probabilistica utilizza Bayesian Ridge. Il modello permette di ottenere una predizione puntuale e, tramite deviazione standard, un intervallo di confidenza associato alla stima. Nel flusso finale questo intervallo viene mostrato insieme alle predizioni degli altri modelli.

4.1 Funzionamento Regressione Bayesiana

La Bayesian Ridge Regression estende la regressione lineare introducendo un approccio bayesiano nella stima dei coefficienti. Invece di stimare un solo valore deterministico per ogni coefficiente, il modello assume una distribuzione a priori e la aggiorna con i dati osservati. In questo modo la predizione puo' essere accompagnata da una misura di incertezza.

$$Y = X * \text{beta} + \text{errore}$$

$$P(\text{beta} \mid D) = P(D \mid \text{beta}) * P(\text{beta}) / P(D)$$

4.2 Iperparametri con Grid Search e Cross Validation

Come nella documentazione originale, il modello viene ottimizzato con Grid Search e Cross Validation a 5 fold. La griglia esplora i parametri `alpha_1`, `alpha_2`, `lambda_1`, `lambda_2` e `fit_intercept`. I risultati della ricerca vengono salvati in formato CSV, mentre i migliori parametri vengono salvati in formato JSON.

Parametro	Valore migliore
<code>alpha_1</code>	0.0001
<code>alpha_2</code>	1e-06
<code>fit_intercept</code>	True
<code>lambda_1</code>	1e-06
<code>lambda_2</code>	0.0001

4.3 Creazione della tabella dei risultati

I risultati della Grid Search del modello Bayesian Ridge vengono salvati in una tabella CSV e i migliori iperparametri vengono esportati in JSON, in continuita' con la struttura della documentazione originale.

4.4 Intervallo di confidenza

L'intervallo di confidenza permette di rappresentare l'incertezza della predizione probabilistica. Nel codice viene calcolato usando la media della predizione e la deviazione standard restituita dal modello.

$$\text{Intervallo} = [\text{media} - 1.96 * \text{sigma}, \text{media} + 1.96 * \text{sigma}]$$

4.5 Valutazione del modello

Metrica	Significato
MAE	Errore medio assoluto tra valori reali e predetti.
RMSE	Radice dell'errore quadratico medio, penalizza gli errori grandi.
R^2	Quota di variabilita' spiegata dal modello.

Nel progetto originale i valori di train e test erano molto vicini, indicando buona capacita' di generalizzazione e assenza di overfitting rilevante per il modello Bayesian Ridge.

4.6 Grafici

4.7 Stime Calorie Bruciate

Nel programma principale la stima probabilistica viene mostrata come valore singolo e come intervallo di confidenza. Questo rende il modello Bayesian Ridge utile non solo per predire le calorie, ma anche per comunicare l'incertezza associata alla stima.

4.8 Modello SGDClassifier fallimentare

La documentazione originale riportava anche un tentativo con SGDClassifier. Il modello, basato su Stochastic Gradient Descent, e' stato scartato perche' non ha raggiunto prestazioni soddisfacenti. Nonostante l'uso di SMOTE per bilanciare le classi e la ricerca degli iperparametri, il modello non ha generalizzato bene e si e' dimostrato meno adatto rispetto alla regressione bayesiana per l'obiettivo del progetto.

4.8.1 Conclusione

La conclusione rimane coerente con la documentazione originale: per questo problema di stima numerica, Bayesian Ridge risulta piu' adatto di SGDClassifier, che nasce per un'impostazione classificatoria e non ha prodotto risultati soddisfacenti.

5. Rappresentazione della Conoscenza - KB

La Knowledge Base si trova in kb/kb.pl e contiene fatti e regole per calcolare calorie per kg, BMI, livello fitness, workout consigliato, intensita' e durata ottimale. Python interroga Prolog tramite pyswip.

Predicato	Funzione
calories_per_kg/2	Associa a ogni workout un consumo stimato per kg e ora.
bmi/3	Calcola l'indice di massa corporea.
fitness_level/2	Classifica il BMI in categorie.
recommended_workout/3	Suggerisce il workout in base a peso e altezza.
recommended_intensity/4	Stima l'intensita' in base alle calorie bruciate.
optimal_duration/4	Calcola una durata ottimale suggerita.

5.1 Struttura della KB

La struttura della KB e' composta da fatti, che rappresentano conoscenza statica, e da regole, che permettono di derivare nuove informazioni a partire dai dati di input dell'utente.

5.1.1 Fatti

I fatti della KB rappresentano conoscenze statiche: consumo calorico per tipo di workout, soglie di intensita', associazione tra livello fitness e workout, durata suggerita in base all'intensita'.

```
calories_per_kg(0, 8.0). % Cardio
calories_per_kg(1, 10.0). % HIIT
calories_per_kg(2, 6.0). % Strength Training
calories_per_kg(3, 3.5). % Yoga

intensity_threshold(high, 800).
intensity_threshold(moderate, 400).

workout_type(underweight, 2).
workout_type(normal, 0).
workout_type(overweight, 1).
workout_type(obese, 3).
```

5.1.2 Regole

Le regole combinano i fatti per derivare nuove conclusioni. La KB non recupera semplicemente un valore salvato: calcola il BMI, determina il livello fitness, sceglie il workout e stima intensita' e durata ottimale attraverso una catena di inferenze.

5.2 Ragionamento Inferenziale nella KB

Il ragionamento inferenziale procede per passaggi: prima viene calcolato il BMI, poi viene individuato il livello di fitness, quindi viene scelto il workout consigliato e infine vengono stimate intensita' e durata ottimale. Questo schema rende esplicita la catena logica seguita dalla Knowledge Base.

```
recommended_workout(Weight, Height, Workout_Type) :-  
    bmi(Weight, Height, BMI),  
    fitness_level(BMI, Level),  
    workout_type(Level, Workout_Type).  
  
recommended_intensity(Weight, Height, Intensity, Duration) :-  
    recommended_workout(Weight, Height, Workout_Type),  
    calories_burned(Workout_Type, Duration, Weight, Calories),  
    intensity_category(Calories, Intensity).
```

5.3 Considerazioni finali

La KB non sostituisce i modelli di machine learning: lavora in modo complementare. I modelli stimano le calorie dai dati, mentre Prolog formalizza regole leggibili e spiegabili per consigliare allenamento, intensita' e durata.

5.4 Esempio pratico

```
?- recommended_workout(70, 1.75, Workout).  
Workout = 0.  
  
?- recommended_intensity(70, 1.75, Intensity, 1.0).  
Intensity = moderate.
```

6. Modulo principale predict_models.py

Il file predict_models.py e' ora il punto di ingresso dell'intera applicazione. Rispetto alla versione precedente, non si limita piu' a stimare le calorie bruciate: dopo le predizioni richiama anche il modulo nutrition_planner.py e stampa il piano alimentare.

- 1 Carica modelli ML, scaler e mappatura dei workout.
- 2 Raccoglie input utente, allenamento e composizione corporea.
- 3 Interroga Prolog per workout, intensita' e durata ottimale.
- 4 Scala i dati e produce le predizioni dei modelli.
- 5 Genera piano giornaliero, alternative o piano settimanale tramite il dataset alimenti.

La versione aggiornata introduce controlli sugli input: sesso ammesso solo come 0 o 1, eta' in un intervallo realistico, peso e altezza positivi, durata della sessione controllata, battiti medi entro range plausibile, massa grassa e massa magra coerenti. Se un valore non e' valido, il programma non prosegue e richiede nuovamente il dato.

```
python3 predict_models.py
```

7. Piano alimentare e nutrition_planner.py

Il modulo nutrition_planner.py riceve dati fisici e di allenamento e trasforma tali informazioni in fabbisogno calorico, macronutrienti e pasti. La logica è deterministica, leggibile e modificabile: questo permette di spiegare chiaramente il processo all'interno del progetto.

7.1 Input richiesti

- sesso: 0 per maschio, 1 per femmina;
- età;
- peso in kg;
- altezza in metri;
- ore di allenamento settimanali;
- percentuale di massa grassa;
- massa magra in kg, oppure 0 se non nota;
- numero di giorni da pianificare, da 1 a 7;
- numero di alternative giornaliere, da 1 a 3, quando si sceglie un solo giorno.

7.2 Formule principali

Calcolo	Formula o regola
Massa magra stimata	$\text{peso} * (1 - \text{massa_grassa_pct} / 100)$, se non inserita.
BMR	$370 + 21.6 * \text{massa_magra}$, formula Katch-McArdle.
TDEE	$\text{BMR} * \text{fattore_attivita}$.
Dimagrimento	$\text{target_calories} = \text{TDEE} * 0.85$.
Aumento massa	$\text{target_calories} = \text{TDEE} * 1.10$.
Mantenimento	$\text{target_calories} = \text{TDEE}$.

7.3 Generazione dei pasti

Il piano viene distribuito su quattro pasti: colazione, pranzo, spuntino e cena. Per ogni pasto il modulo seleziona alimenti compatibili con il pasto e con la categoria nutrizionale richiesta, poi calcola i grammi in base al budget calorico associato a proteine, carboidrati e grassi. Per variare le risposte, il sistema usa un indice di variante che cambia gli alimenti selezionati. Se il piano è settimanale, ogni giorno usa una variante diversa.

La selezione degli alimenti avviene filtrando il dataset per pasto e categoria nutrizionale. Ad esempio, per la colazione vengono scelti alimenti compatibili con il campo pasti contenente colazione; per pranzo e cena vengono considerati alimenti compatibili con pranzo o cena. L'indice di variante consente di scorrere alimenti diversi senza cambiare la struttura del piano.

Pasto	Quota calorica
Colazione	25%

Pasto	Quota calorica
Pranzo	35%
Spuntino	10%
Cena	30%

7.4 Bilanciamento del piano

Dopo la generazione dei pasti il modulo calcola i totali giornalieri e, se necessario, aggiunge una piccola quota calorica usando alimenti densi come fonti di carboidrati o grassi. Questo passaggio permette di avvicinare il piano al target calorico senza stravolgere la composizione dei pasti.

Funzione	Ruolo
<code>select_food</code>	Sceglie un alimento coerente con pasto e categoria.
<code>calculate_food_grams_by_calories</code>	Converte il budget calorico in grammi.
<code>calculate_plan_totals</code>	Somma calorie e macronutrienti del giorno.
<code>balance_daily_plan</code>	Bilancia il piano aggiungendo alimenti se il totale e' troppo distante dal target.
<code>generate_weekly_meal_plan</code>	Ripete la generazione su piu' giorni usando varianti diverse.

7.5 Esempio di output

```

=== Piano Alimentare Consigliato ===
Obiettivo stimato: mantenimento
Calorie giornaliere consigliate: 2481.86 kcal
Proteine: 102.60 g | Carboidrati: 376.12 g | Grassi: 63.00 g

Colazione
- Yogurt greco 0%: 174 g
- Avena: 97 g
- Mandorle: 24 g

```

7.6 Piano settimanale

Se l'utente sceglie 7 giorni, il programma stampa un piano alimentare settimanale da Lunedì a Domenica. Ogni giorno riporta totali stimati di calorie, proteine, carboidrati e grassi, seguiti dai quattro pasti con alimenti e grammature. Il piano viene bilanciato per restare vicino al target calorico giornaliero.

7.7 Limiti e possibili estensioni

Il piano alimentare e' pensato come output informativo del progetto. Non tiene ancora conto di allergie, patologie, preferenze alimentari, fasce orarie, budget economico o vincoli culturali. Questi aspetti possono essere aggiunti estendendo il dataset alimenti.csv e inserendo nuovi filtri nella funzione di generazione dei pasti.

8. Dipendenze e Documentazione

Per testare il programma completo dalla cartella principale del progetto si esegue il comando seguente. L'output atteso contiene tre blocchi: inferenza Prolog, predizioni ML e piano alimentare.

```
cd /Users/christiandomenicotermine/Desktop/Burned-Calories-Estimation-Model-main
python3 predict_models.py
```

8.1 Esecuzione e test

L'esecuzione principale avviene da terminale. Durante i test sono state verificate sia la generazione del piano settimanale sia la generazione di piu' alternative giornaliere, oltre al comportamento dei controlli sugli input.

8.2 Aggiornamento dei grafici

I grafici e i modelli non vengono aggiornati da predict_models.py. Per rigenerarli e' necessario rilanciare gli script di training.

```
python3 apprendimento_supervisionato/training.py
python3 apprendimento_probabilistico/training.py
```

8.3 Documentazione HTML

La cartella docs contiene i sorgenti Sphinx aggiornati. Per evitare conflitti con le versioni di numpy su Python 3.9, le dipendenze della documentazione sono state separate in requirements-docs.txt. Questo permette di generare l'HTML senza reinstallare tutte le dipendenze ML.

```
python3 -m pip install -r requirements-docs.txt
python3 -m sphinx -b html docs docs/_build/html
```

8.4 Documentazione in Prolog

Come nella documentazione originale, anche la Knowledge Base Prolog puo' essere documentata separatamente. La documentazione generata da Prolog si trova nella cartella kb/doc e descrive i predicati definiti nel file kb.pl.

```
kb/doc/index.html
kb/doc/kb.html
```

8.5 Dipendenze principali

Per eseguire il progetto servono Python, SWI-Prolog e le librerie Python elencate in requirements.txt. Per la sola documentazione HTML servono Sphinx e sphinx-rtd-theme, ora collocati in requirements-docs.txt.

8.6 Installazione delle Dipendenze

L'installazione segue la stessa logica della relazione originale: prima si prepara Python, poi si installa SWI-Prolog per usare pyswip, infine si installano le librerie Python. Nel progetto aggiornato le dipendenze della documentazione HTML sono state separate da quelle del modello.

```
python3 -m pip install -r requirements.txt
python3 -m pip install -r requirements-docs.txt
```

9. Considerazioni finali

La versione aggiornata del progetto mantiene la parte originale di stima delle calorie e la amplia con un sistema di raccomandazione alimentare. La scelta di separare il modulo nutrizionale in `nutrition_planner.py` rende il codice piu' leggibile, facilita l'estensione del dataset alimenti e permette di distinguere chiaramente predizione, inferenza e pianificazione nutrizionale.

Il piano alimentare generato deve essere inteso come output informativo e progettuale. In contesti reali, condizioni cliniche, allergie o obiettivi sportivi specifici richiedono la valutazione di un professionista della nutrizione.

10. Riferimenti Bibliografici

- Dataset Kaggle: Gym Members Exercise Tracking.
- Documentazione scikit-learn per modelli di regressione e GridSearchCV.
- Documentazione pandas per gestione di dataset CSV.
- Documentazione pyswip per integrazione Python-Prolog.
- Sorgenti locali del progetto aggiornati al 24 giugno 2026.

Scheda 1 - Obiettivo generale del progetto

La relazione originale partiva dalla stima delle calorie bruciate durante una sessione di allenamento. Nel progetto aggiornato questo obiettivo rimane centrale, ma diventa il primo passo di un flusso più completo: le calorie stimate vengono usate insieme a composizione corporea, età, sesso, peso, altezza e ore di allenamento per costruire un piano alimentare coerente con il fabbisogno dell'utente.

Scheda 2 - Dataset di allenamento

Il dataset `gym_members_exercise_tracking.csv` mantiene il ruolo originale: descrive utenti, sessioni di allenamento, parametri fisiologici e calorie bruciate. La variabile target resta `Calories_Burned`, mentre le feature usate nel programma principale sono quelle salvate nello scaler e coerenti con il training dei modelli.

Scheda 3 - Preprocessing e mapping

Il preprocessing rinomina colonne, converte Gender in valori numerici e trasforma Workout_Type in categoria. Il mapping dei workout viene salvato su disco per mostrare nuovamente all'utente i nomi leggibili degli allenamenti. La standardizzazione con StandardScaler garantisce che i modelli ricevano input nella stessa scala usata in addestramento.

Scheda 4 - Regressione Lineare

La Regressione Lineare rimane una baseline utile per interpretare il problema. Offre un confronto semplice rispetto ai modelli ensemble: se l'errore è alto, significa che la relazione tra variabili e calorie bruciate non è solo lineare. Per questo nella relazione vengono poi analizzati Random Forest e Gradient Boosting.

Scheda 5 - Random Forest

Random Forest usa piu' alberi decisionali e combina le loro predizioni. La Grid Search permette di provare configurazioni diverse di profondita', numero di stimatori e soglie minime di split. La relazione evidenzia anche il rischio di overfitting quando le prestazioni sul training set sono migliori rispetto al test set.

Scheda 6 - Gradient Boosting

Gradient Boosting costruisce gli alberi in sequenza e corregge progressivamente gli errori residui. Nel progetto originale era uno dei modelli più efficaci; nella versione aggiornata resta nel confronto delle predizioni e viene caricato da file .pkl insieme agli altri modelli supervisionati.

Scheda 7 - Grafici dei modelli supervisionati

I grafici presenti nelle cartelle apprendimento_supervisionato/grafici non vengono rigenerati durante l'esecuzione di predict_models.py. Vengono aggiornati solo rilanciando lo script di training. Questa separazione evita che una predizione utente modifichi accidentalmente i risultati sperimentali del progetto.

Scheda 8 - Bayesian Ridge

Bayesian Ridge mantiene la parte probabilistica della relazione originale. Il vantaggio rispetto a una regressione standard e' la possibilita' di associare alla predizione una misura di incertezza. Nel programma questa informazione viene stampata come intervallo di confidenza.

Scheda 9 - SGDClassifier scartato

La documentazione originale riportava il tentativo con SGDClassifier. La nuova relazione conserva questa informazione per completezza metodologica: il modello non era adeguato al problema finale di stima numerica delle calorie, quindi la scelta progettuale si concentra su regressione e Bayesian Ridge.

Scheda 10 - Knowledge Base Prolog

La KB in kb.pl rende esplicite regole interpretabili: calcolo del BMI, classificazione del livello fitness, scelta del workout, stima dell'intensita' e durata ottimale. Questa componente lavora accanto ai modelli ML, aggiungendo spiegabilita' al flusso.

Scheda 11 - Query Prolog nel codice

`predict_models.py` interroga Prolog con `recommended_workout`, `recommended_intensity` e `optimal_duration`. Il cambio temporaneo di `directory` verso `kb` serve a evitare problemi di caricamento della libreria `pyswip` e della base di conoscenza Prolog.

Scheda 12 - Dataset alimentare

Il file `dataset/alimenti.csv` contiene alimenti, categorie, pasti compatibili e macronutrienti per 100 grammi. Non e' un file Excel nativo, ma puo' essere aperto con Excel. Il planner usa questo dataset per selezionare alimenti coerenti con il pasto e con il budget calorico del giorno.

Scheda 13 - Calcolo nutrizionale

nutrition_planner.py calcola massa magra, BMR, TDEE, obiettivo calorico e macronutrienti. Se la massa magra non e' nota, l'utente puo' inserire 0 e il programma la stima usando peso e percentuale di massa grassa.

Scheda 14 - Alternative giornaliere

Quando l'utente sceglie un solo giorno puo' richiedere piu' alternative. Il codice usa un indice di variante per cambiare gli alimenti selezionati mantenendo la stessa logica di calcolo delle calorie e dei macronutrienti.

Scheda 15 - Piano settimanale

Il piano settimanale estende la generazione fino a sette giorni, da Lunedì a Domenica. Ogni giorno mantiene la divisione in colazione, pranzo, spuntino e cena, ma usa varianti diverse per evitare un output troppo ripetitivo.

Scheda 16 - Validazione degli input

Gli input sono controllati prima di essere usati: sesso ammesso 0 o 1, età e peso entro range realistici, altezza positiva, battiti medi plausibili, massa grassa e massa magra coerenti. Se un dato non è valido, il programma ripete la domanda.

Scheda 17 - Documentazione e GitHub

La documentazione aggiornata comprende PDF, HTML navigabile, sorgenti Sphinx e documentazione Prolog. La build HTML contiene pagine dedicate a indice originale, moduli, dataset, KB, planner nutrizionale e output. Il repository GitHub deve contenere codice, dataset, documentazione e file requirements, escludendo cache Python e file di sistema.